

Arquitectura del Sistema

Introduccion

Este documento detalla la arquitectura del sistema Syshub, diseñado para centralizar y gestionar recursos academicos, foros de discusion y la administracion de usuarios. El enfoque arquitectonico se basa en un modelo cliente-servidor desacoplado, utilizando tecnologias modernas que aseguran escalabilidad, mantenibilidad y alto rendimiento.

Patron Arquitectonico General

El sistema esta compuesto por tres capas principales:

1. Capa de Presentacion (Frontend): Desarrollada en Vue.js v3 utilizando Vite como herramienta de construccion y Tailwind CSS para el diseno de interfaz.
2. Capa de Logica de Negocio (Backend): Construida sobre NestJS, un framework para construir aplicaciones Node.js eficientes y escalables del lado del servidor.
3. Capa de Persistencia de Datos (Base de Datos): Utiliza PostgreSQL como sistema de gestion de bases de datos relacional.

Integracion Frontend - Backend

La comunicacion entre el frontend y el backend se realiza mediante una API RESTful stateless. Las peticiones HTTP (GET, POST, PUT, DELETE, PATCH) son interceptadas por el backend, procesadas a traves de controladores y delegadas a los servicios correspondientes.

Capa Frontend (Detalle de Archivos)

El frontend sigue la arquitectura de componentes y estado global de Vue.js:

- `src/main.ts`: Punto de entrada de la aplicacion Vue. Configura Pinia, Vue Router y el componente raiz `App.vue`.
- `src/App.vue`: Contenedor principal que maneja transiciones y renderiza el Layout y el router-view.
- `src/router/index.ts`: Define las rutas de navegacion. Protege rutas especificas garantizando que solo usuarios autenticados (y con roles apropiados) accedan a ciertas vistas (ej. panel de administracion).
- `src/stores/`: Utiliza Pinia para la gestion del estado. `authStore.ts` maneja la sesion del usuario, el token JWT y los roles, verificando permisos como `isModerator` o `isAdmin`.
- `src/api/`: Contiene abstracciones de axios (ej. `auth.ts`, `moderation.ts`, `admin.ts`) para realizar peticiones HTTP al backend, estableciendo cabeceras de autorizacion (Bearer tokens) automaticamente.
- `src/views/`: Vistas principales (`Projects.vue`, `Forums.vue`, `Admin.vue`) que consumen los componentes y los stores para renderizar la informacion obtenida.

Capa Backend (Detalle de Archivos)

El backend esta organizado en modulos logicos definidos por NestJS:

- `src/main.ts`: Punto de arranque de la API. Configura middlewares, validacion global (`ValidationPipe`) y el prefijo global (opcional).
- `src/app.module.ts`: Modulo raiz que importa TypeORM, configuraciones de variables de entorno y los distintos modulos (`AuthModule`, `ProjectsModule`, `ForumsModule`, etc.).
- `src/config/typeorm.config.ts`: Centraliza la conexion a PostgreSQL y la carga de entidades.
- `src/modules/`: Cada modulo encapsula su dominio:

- AuthModule: Gestiona la logica de inicio de sesion, registro y emision de tokens JWT. Emplea guards (JwtAuthGuard, RolesGuard) para proteger endpoints.
- ProjectsModule, ForumsModule, ArticlesModule: Gestionan los repositorios de informacion academica y discusiones. Exponen servicios apoyados en TypeORM Repository o consultas crudas.
- ModerationModule: Gestiona la capacidad de ocultar contenido o vetarlo, integrando permisos de moderador y guardando trazabilidad en la tabla de reportes.
- AdminModule: Interfaz orientada a superadministradores para moderar contenido, asignar roles y aprobar usuarios pendientes.

Integracion con la Base de Datos

TypeORM actua como la capa ORM (Object-Relational Mapping), pero para modulos complejos (como Moderation) se realizan inyecciones de consultas crudas (raw queries) garantizando desempeno en actualizaciones de alto impacto. La base de datos, estructurada segun los schemas en database/01_schema.sql, aloja tablas fuertemente tipadas y referenciadas.

Despliegue y Contenedorizacion

Todo el entorno esta orquestado mediante Docker Compose, incluyendo docker-compose.yml en la raiz y sub-Dockerfiles en frontend/ y backend/.

- backend/Dockerfile: Transpila TypeScript a Javascript y ejecuta el entorno en node.
- frontend/Dockerfile: Compila el bundle de produccion mediante Vite o levanta un entorno de dev. Esto estandariza la infraestructura de desarrollo y produccion, disminuyendo discrepancias entre entornos.